

VERIQTA



Network Engineering Guidebook



**NETWORK
FUNDAMENTALS**



**PROTOCOLS
AND SERVICES**



SECURITY



**CLOUD
NETWORKING**



TROUBLESHOOTING



**REAL-WORLD
PRACTICE**

01 How Computer Networks Work

What is a Computer Network?

A computer network is a group of devices connected together so they can share data, resources and services.

Why Do We Use Networks?

- Share files and services
- Access the internet
- Communicate between systems
- Run applications for users
- Connect offices, data centres and cloud

How a Network Connection Works

When you access a website or an application, your device sends data to another system over a network. The data is broken into small pieces called packets.



Host



Any device connected to a network. Examples: laptop, phone, server, printer, IoT device.

Client



A device that requests a service or resource from another system.

Server



A device or application that provides a service or resource to clients.

Packet



Small piece of data that travels across networks.

LAN (Local Area Network)



A network in a small geographic area such as a home, office or building.

WAN (Wide Area Network)



A network that covers a large geographic area such as a city, country or continent.

Internet



A global network of networks that connects millions of devices worldwide.

Real-World Example

When you open a website:

- 1 Your device sends a request.
- 2 The request travels through your network, the internet, and reaches the website server.
- 3 The server sends a response, and your device displays the website.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

01

02

The OSI and TCP/IP Models

Why Do We Use Models?

- Models break networking into layers to make it easier to understand.
- Each layer has a specific job.
- They help engineers design, configure, and troubleshoot networks.

Two Popular Models

- OSI Model: 7 layers.**
Used for learning and reference.
- TCP/IP Model: 4 layers.**
Used in real networks (Internet, Linux, Windows, cloud, etc.).

OSI Model (7 Layers)

7	Application	Provides network services to user applications.
6	Presentation	Formats, encrypts and compresses data.
5	Session	Manages communication sessions.
4	Transport	Provides end-to-end communication (TCP/UDP).
3	Network	Handles logical addressing and routing (IP).
2	Data Link	Handles local delivery using MAC addresses.
1	Physical	Transmits raw bits over physical media (cables, Wi-Fi).

TCP/IP Model (4 Layers)

Application	Combines OSI layers 7, 6 and 5. Example: HTTP, DNS, FTP, SSH.
Transport	Provides end-to-end communication. Example: TCP, UDP.
Internet	Handles addressing and routing. Example: IP, ICMP.
Network Access	Combines OSI layers 2 and 1. Example: Ethernet, Wi-Fi, switches.

How Data Flows (Example: You Open a Website)



- Application:** You type `www.example.com` in your browser.
- Transport:** TCP creates a connection.
- Internet:** IP routes the packets to the server.
- Network Access:** Data is sent over Ethernet or Wi-Fi.
- The server responds and the data travels back through the same layers.

How Engineers Use the Layers for Troubleshooting

- Application:** Is the application working? (e.g. HTTP 200 or 404)
- Transport:** Is the port open? (e.g. TCP connection succeeds)
- Network:** Can the destination IP be reached? (e.g. ping, traceroute)
- Data Link:** Is the device on the same network? (e.g. ARP, switch)
- Physical:** Is the cable or Wi-Fi working? (e.g. link lights, signal strength)

Key Takeaway



Each layer depends on the layer below it. If something fails, check the lower layers first.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

02

03 IP Addresses Explained

What is an IP Address?

An IP address is a unique number that identifies a device on a network so it can send and receive data. It works like a home address, but for computers.

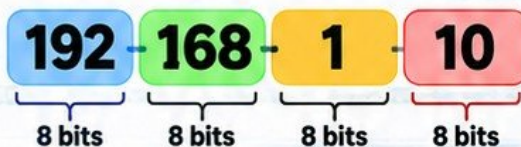


Why Do Machines Need IP Addresses?

- To identify each device
- To send data to the correct device
- To communicate across networks
- To access the internet
- To run applications and services

IPv4 Address Format

An IPv4 address has 32 bits, written as four numbers between 0 and 255.



IPv6 Address Format

IPv6 uses 128 bits and supports many more devices. It is written in hexadecimal.

2001:0db8:85a3:0000:0000:8a2e:0370:7334

Used because IPv4 addresses are limited.

Public vs Private IP Addresses

Private IP (Internal Use)

- Used in homes, offices and internal networks
- Not accessible directly from the internet
- Common ranges:

10.0.0.0 – 10.255.255.255

172.16.0.0 – 172.31.255.255

192.168.0.0 – 192.168.255.255

Public IP (Internet Use)

- Used on the internet
- Globally unique
- Assigned by your ISP or cloud provider
- Accessible from anywhere on the internet



Network vs Host Portion

An IP address is divided into a network part and a host part using a subnet mask or CIDR.

192.168.1.10/24

192.168.1

Network Portion
(24 bits)

10

Host Portion
(8 bits)

Common Special IP Addresses

Loopback	127.0.0.1	Refers to your own device (Localhost). Used for testing.
Any Address	0.0.0.0	Used to listen on all network interfaces.
Broadcast	192.168.1.255	Sends data to all devices in the network (for that subnet).
Unspecified	0.0.0.0	Used when a device does not yet have an IP address.

Real-World Example



Your device uses a private IP inside your network. The router uses NAT to translate it to a public IP so you can access the internet.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

03

04 Subnets and CIDR

What are Subnets?

- A subnet (subnetwork) is a smaller network created from a larger network.
- It divides IP addresses into smaller ranges.
- Each subnet has its own network address, host range, and broadcast address.

Subnet Mask

A subnet mask separates the network portion and the host portion of an IP address.

192.168.1.10 / 255.255.255.0

IP Address

Subnet Mask

Example: /24 Subnet

IP Address: 192.168.1.0/24
Subnet Mask: 255.255.255.0

Network Address	192.168.1.0
Usable Host Range	192.168.1.1 - 192.168.1.254
Broadcast Address	192.168.1.255
Total Addresses	256 (254 usable hosts)

Common CIDR Subnet Sizes

CIDR	Subnet Mask	Total IPs	Usable Hosts
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14

What is CIDR Notation?

- CIDR (Classless Inter-Domain Routing) is a shorter way to write a subnet mask.
- It uses a slash (/) followed by the number of network bits.
- Example: 192.168.1.0/24 means the first 24 bits are for the network.

Network vs Host Portion

In 192.168.1.10/24:



Example: /26 Subnet

IP Address: 192.168.1.0/26
Subnet Mask: 255.255.255.192

Network Address	192.168.1.0
Usable Host Range	192.168.1.1 - 192.168.1.62
Broadcast Address	192.168.1.63
Total Addresses	64 (62 usable hosts)

Why Engineers Subnet Networks?

- Better use of IP addresses
- Separates departments and services
- Improves security (isolation)
- Reduces broadcast traffic
- Easier network management
- Required in large networks and cloud environments

Key Takeaway

 Subnets help you divide a large network into smaller, manageable networks using subnet masks or CIDR notation.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

04

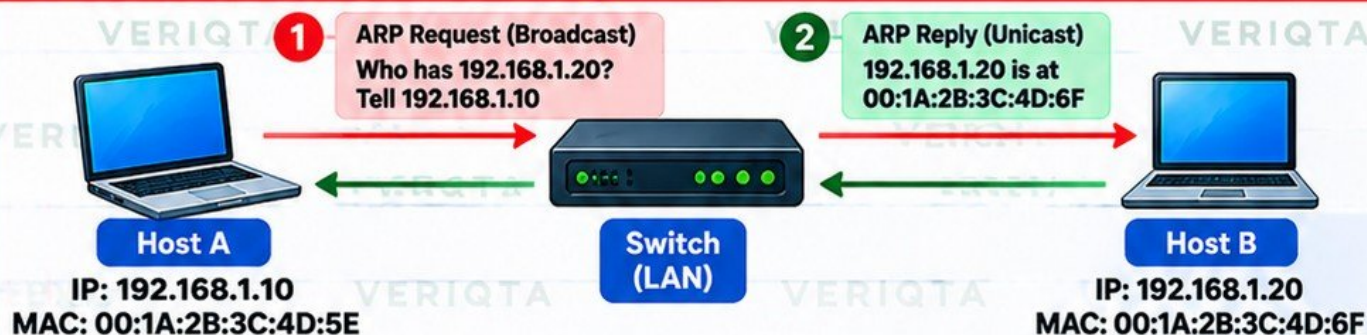
What is a MAC Address?

- A MAC address (Media Access Control address) is a unique hardware address assigned to a network interface.
- It is 48 bits written in hexadecimal (e.g. 00:1A:2B:3C:4D:5E).
- Used for local delivery on a LAN (Ethernet or Wi-Fi).
- Each device has a unique MAC address (usually assigned by the manufacturer).

What is ARP?

- ARP (Address Resolution Protocol) finds the MAC address for a given IPv4 address on the local network.
- It is used when a device wants to send data to another device on the same LAN.
- ARP broadcasts a request and the device with the matching IP replies with its MAC address.

How ARP Works (Local Delivery)



ARP Table (ARP Cache)

Devices store IP to MAC mappings in a temporary ARP table (cache).

IP Address	MAC Address
192.168.1.1	00:1A:2B:3C:4D:01
192.168.1.20	00:1A:2B:3C:4D:6F
192.168.1.50	00:1A:2B:3C:4D:99

Useful Commands (Linux)

View ARP table (modern)

```
ip neigh show
```

View ARP table (older)

```
arp -n
```

Send ARP request manually

```
arping -I eth0 192.168.1.20
```

Clear ARP entry

```
ip neigh del 192.168.1.20
```

Key Points

- MAC addresses are used only on the local network.
- ARP helps map an IP address to a MAC address.
- ARP uses broadcast for requests and unicast for replies.
- Switches forward frames using MAC addresses.

Common Scenarios

- 1 ARP table is empty → host sends a broadcast request.
- 2 Target device replies with its MAC address.
- 3 Host stores the mapping in its ARP table.
- 4 Data is then sent directly to the target MAC address at Layer 2.

Troubleshooting Tips

- If a device is not reachable on the LAN, check the ARP table.
- Clear stale ARP entries.
- Verify the device is on the same subnet.
- Use `arping` to test if the device replies.

Real-World Use

- Your laptop uses ARP to find the MAC address of your router.
- Servers use ARP to communicate with other devices on the same network.
- ARP is essential for local network communication.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

05

06 Switches, Routers and Gateways

What is a Switch?



Switch

- A switch connects devices within the same network (LAN).
- Works at Layer 2 (Data Link Layer).
- Forwards frames using MAC addresses.
- Each port is a separate collision domain.
- Commonly used in homes, offices and data centres.

What is a Router?



Router

- A router connects different networks (e.g. your LAN to the internet).
- Works at Layer 3 (Network Layer).
- Forwards packets using IP addresses.
- Chooses the best path to reach destination networks.
- Often performs NAT, firewalling and routing between networks.

What is a Gateway?



- A gateway is the device that connects your local network to other networks (such as the internet).
- In most networks, the router is the default gateway.
- Devices send traffic to the gateway when the destination IP is outside the local network.

Layer 2 vs Layer 3 Forwarding

Layer 2 (Switch)	Layer 3 (Router)
Uses MAC addresses	Uses IP addresses
Forwards frames within the same network (LAN)	Forwards packets between different networks
Does not change IP addresses	Can change IP addresses (e.g. NAT)
Works at the Data Link layer	Works at the Network layer
Example device: Switch	Example device: Router

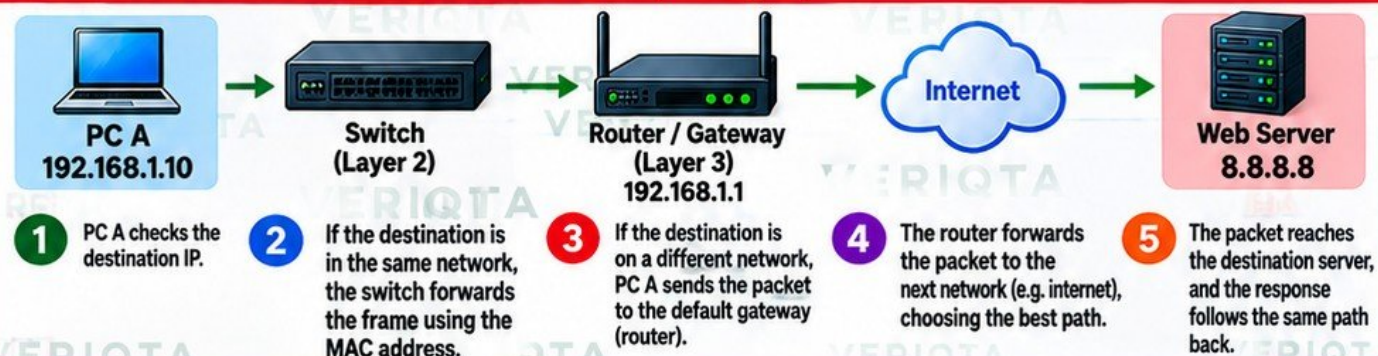
Default Gateway

The default gateway is the IP address of the router that your device uses to reach other networks (such as the internet).

Example:

- Your laptop IP: 192.168.1.10
- Subnet mask: 255.255.255.0
- Default gateway: 192.168.1.1
- Traffic to 192.168.1.x stays in the local network.
- Traffic to other networks (e.g. 8.8.8.8) is sent to the default gateway.

Packet Movement Between Networks (Example)



Key Takeaways

- Switches connect devices in the same network using MAC addresses (Layer 2).
- Routers connect different networks using IP addresses (Layer 3).
- The default gateway is usually your router's IP address.
- Packets to a different network are sent to the default gateway.
- Switches forward frames, routers forward packets.

Common Devices and Layers

Device	Layer	Uses	Example
Switch	Layer 2	MAC addresses	Office switch
Router	Layer 3	IP addresses	Home/Office router
Gateway	Layer 3	Connects networks	Router (192.168.1.1)



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

06

07

Routing Fundamentals

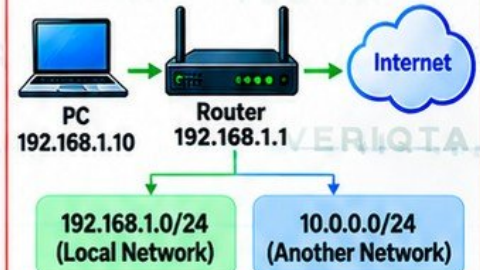
What is Routing?

- Routing is the process of forwarding packets from one network to another.
- Routers use routing tables to decide the best path to the destination.
- Each route tells the router where to send packets next (next hop).

Key Terms

- **Routing Table:** List of routes.
- **Route:** Path to a destination network.
- **Next Hop:** The next router IP.
- **Default Route:** Used when no specific route exists.
- **Longest Prefix Matching:** Router chooses the most specific matching route.

Simple Network Example



Example Routing Table

Destination	Gateway (Next Hop)	Interface	Meaning
192.168.1.0/24	-	eth0	Local network (directly connected)
10.0.0.0/24	192.168.1.254	eth0	Send to next hop 192.168.1.254
0.0.0.0/0	192.168.1.1	eth0	Default route (use this for any unknown network)

Longest Prefix Matching

When multiple routes match, the router chooses the most specific (longest prefix).

10.0.0.0/8	Less specific
10.1.0.0/16	More specific
10.1.1.0/24	Most specific (used if all exist)

Linux ip route Command

```

# Show routing table
ip route

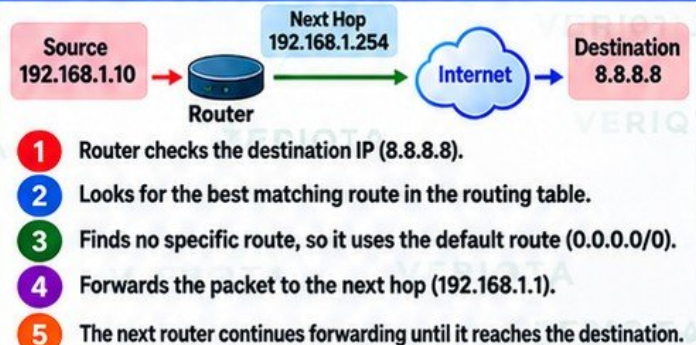
# Show detailed information
ip route show

# Show route to a specific destination
ip route get 8.8.8.8

# Add a static route
sudo ip route add 10.0.0.0/24 via 192.168.1.254

# Delete a route
sudo ip route del 10.0.0.0/24
  
```

How a Router Forwards a Packet



Common Route Types

Route Type	Destination	Example	Used For
Local	Network on same interface	192.168.1.0/24	Directly connected networks
Remote	Network on another router	10.0.0.0/24 via 192.168.1.254	Reach other networks
Default	Any destination	0.0.0.0/0 via 192.168.1.1	Used when no other route matches
Host	Single IP address	8.8.8.8/32	Route to a specific host

Real-World Example: Multiple Networks



Key Takeaways

- Routers use routing tables to forward packets.
- Next hop is the IP of the next router.
- Default route is used when no specific route matches.
- Longest prefix matching chooses the most specific route.
- Use `ip route` to view and manage routes on Linux.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

07

What is TCP?

- Connection-oriented protocol.
- Reliable delivery of data.
- Uses sequencing and acknowledgements.
- Detects and retransmits lost packets.
- Ensures data arrives in the correct order.
- Used by applications where data accuracy is important.

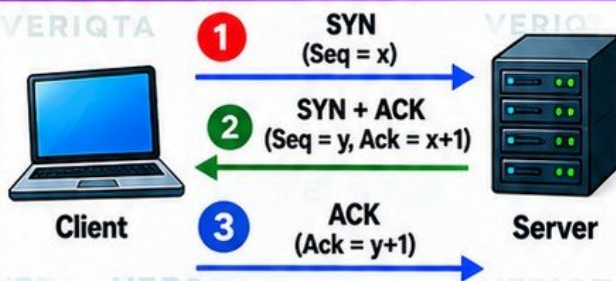
What is UDP?

- Connectionless protocol.
- No guarantee of delivery.
- No sequencing or acknowledgements.
- Does not retransmit lost packets.
- Faster and lower overhead.
- Used by applications where speed is more important than guaranteed delivery.

Key Differences

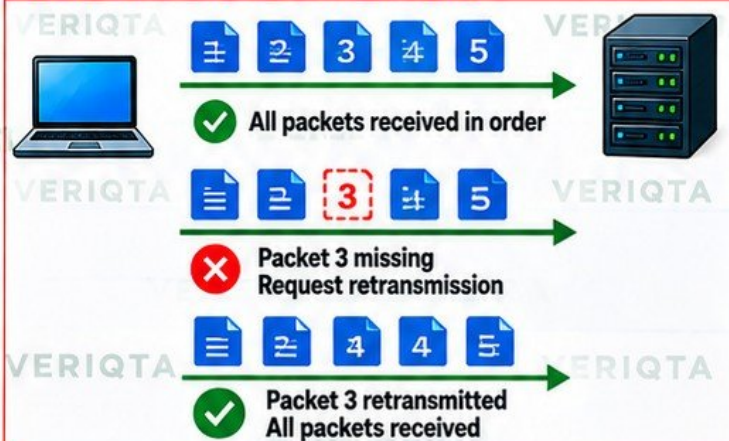
Feature	TCP	UDP
Connection	Yes	No
Reliability	Reliable	No guarantee
Sequencing	Yes	No
Acknowledgement	Yes	No
Retransmission	Yes	No
Speed	Slower	Faster
Overhead	Higher	Lower
Common Uses	Web, Email, File Transfer	Streaming, VoIP, Gaming, DNS

TCP 3-Way Handshake



- 1 Client sends a SYN to start the connection.
- 2 Server replies with SYN + ACK.
- 3 Client sends ACK. Connection is established.

How TCP Ensures Reliability



UDP Behavior



- Sends packets without establishing a connection.
- Does not check if packets are received.
- Packets can arrive out of order.
- Some packets may be lost.
- No retransmission.
- Used where speed matters more than guaranteed delivery.

Real Application Examples

Application	Protocol	Reason
Web Browsing (HTTP/HTTPS)	TCP	Needs reliable delivery
File Transfer (FTP, SFTP)	TCP	Needs complete and ordered data
Email (SMTP, IMAP, POP3)	TCP	Needs reliable delivery
DNS Queries	UDP	Small, fast requests
Video Streaming	UDP	Speed is more important than perfect delivery
VoIP (Voice Calls)	UDP	Low latency, real-time
Online Gaming	UDP	Faster response time
DHCP	UDP	Simple request/response
SNMP	UDP	Lightweight network management

Common Port Numbers

Application	Protocol	Port
HTTP	TCP	80
HTTPS	TCP	443
FTP	TCP	21
SSH	TCP	22
DNS	UDP	53
DHCP	UDP	67/68

Key Takeaways

- TCP is reliable, connection-oriented, and ensures ordered delivery.
- UDP is faster, connectionless, and has no delivery guarantees.
- TCP uses a 3-way handshake to establish a connection.
- TCP uses sequencing and acknowledgements to ensure reliability.
- Use TCP when data accuracy matters, and UDP when speed and low latency are more important.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

08

What are Ports?

- Ports are logical numbers (0–65535) used to identify services on a device.
- An IP address identifies the device, and a port identifies the application (service) on that device.
- Multiple services can run on the same device using different ports.
- Ports work with TCP and UDP.

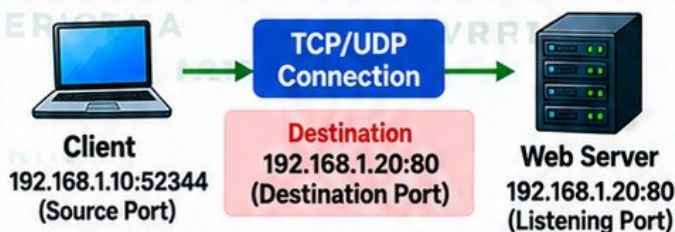
What are Sockets?

- A socket is a combination of an IP address and a port (e.g. 192.168.1.10:80).
- Sockets provide the endpoint for network communication.
- A connection is identified by the source IP and port, and the destination IP and port (4-tuple).

Why Ports Exist?

- Allow multiple applications to use the network at the same time.
- Help the OS deliver traffic to the correct service.
- Common services use well-known ports (e.g. 80 for web, 22 for SSH).
- Custom applications can use any available port.

How Ports and Sockets Work



- Client chooses a random source port (e.g. 52344).
- Client connects to the server IP and destination port (e.g. 80).
- Server listens on the port for incoming connections.
- Data flows between the client and server using the socket (source IP:port ↔ destination IP:port).

Common Well-Known Ports

Service	Protocol	Port	Description
HTTP	TCP	80	Web traffic (not encrypted)
HTTPS	TCP	443	Web traffic (encrypted)
SSH	TCP	22	Secure remote access
FTP	TCP	21	File transfer
SMTP	TCP	25	Send email
DNS	UDP	53	Domain name resolution
DHCP	UDP	67/68	IP address assignment
NTP	UDP	123	Time synchronization
RDP	TCP	3389	Remote desktop (Windows)
MySQL	TCP	3306	MySQL database
PostgreSQL	TCP	5432	PostgreSQL database

Checking Listening Ports (ss)

```
# Show all listening ports
ss -tln

# Show listening TCP ports
ss -tln

# Show listening UDP ports
ss -uln

# Show established connections
ss -tun
```

Example Output

Netid	State	Local Address:Port	Peer Address:Port
tcp	LISTEN	0.0.0.0:22	0.0.0.0:*
tcp	LISTEN	0.0.0.0:80	0.0.0.0:*
tcp	LISTEN	0.0.0.0:443	0.0.0.0:*
udp	UNCONN	0.0.0.0:53	0.0.0.0:*
udp	UNCONN	0.0.0.0:67	0.0.0.0:*

Connection Refused

You get a connection refused error when:

- No service is listening on the destination port.
- The service is down.
- A firewall is blocking the port.
- You are using the wrong IP or port.

```
$ curl 192.168.1.20:8080
curl: (7) Failed to connect to 192.168.1.20 port 8080: Connection refused
```

Source vs Destination Ports

Port Type	Used By	Purpose	Example
Source Port	Client	Temporary, random port	52344
Destination Port	Server	Identifies the service	80 (HTTP)

Key Takeaways



- Ports identify services, IP addresses identify devices.
- A socket is an IP address and a port.
- Servers listen on specific ports.
- Clients use temporary source ports.
- Use ss to check listening ports and active connections.
- Connection refused means no service is listening or the port is blocked.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

09

10

DNS From Name to IP

What is DNS?

- DNS (Domain Name System) translates human readable names (e.g. `www.example.com`) to IP addresses (e.g. `93.184.216.34`).
- It works in a distributed hierarchical system.
- It allows you to use easy to remember names instead of IP addresses.
- DNS is essential for almost every internet service.

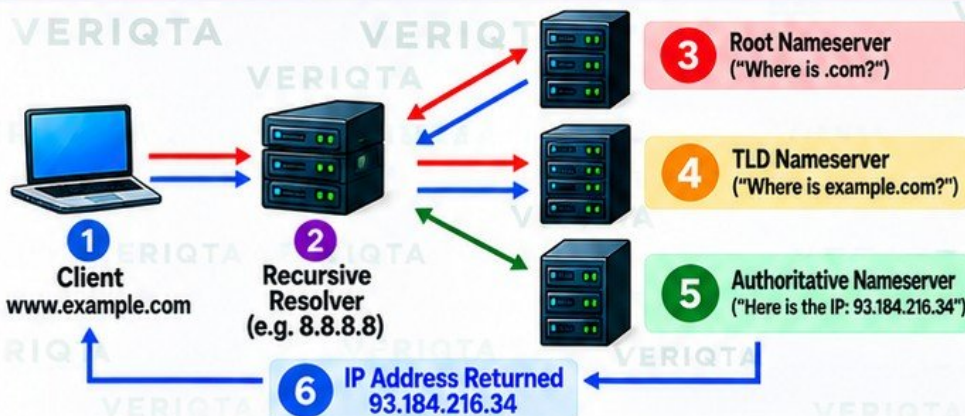
Key Components

- **Client:** Your device (laptop, phone).
- **Recursive Resolver:** Usually provided by your ISP or a public DNS (e.g. `8.8.8.8`).
- **Root Nameserver:** Points to the appropriate TLD nameserver.
- **TLD Nameserver:** For example `.com`, `.org`, `.net`.
- **Authoritative Nameserver:** Holds the actual DNS records for the domain.

Common DNS Record Types

Type	Purpose
A	Maps a domain to an IPv4 address.
AAAA	Maps a domain to an IPv6 address.
CNAME	Alias to another domain name.
MX	Mail server for the domain.
NS	Authoritative nameserver.
TXT	Stores text (used for verification, SPF, etc).
SRV	Service location (host and port).

How DNS Resolution Works (Step by Step)



- 1 Client asks the recursive resolver for `www.example.com`.
- 2 Resolver checks its cache. If not found, it queries a root nameserver.
- 3 Root nameserver returns the `.com` TLD nameserver.
- 4 TLD nameserver returns the authoritative nameserver for `example.com`.
- 5 Authoritative nameserver returns the IP address (A/AAAA record).
- 6 Resolver caches the result and returns the IP to the client.

Using dig (Detailed Query)

```
# Query A record
dig example.com

# Query AAAA record
dig example.com AAAA

# Query specific nameserver
dig example.com @8.8.8.8

# Short output
dig +short example.com
```

Using nslookup (Simple Query)

```
# Basic query
nslookup example.com

# Query specific nameserver
nslookup example.com 8.8.8.8

# Query a specific record type
nslookup -type=AAAA example.com
```

Example Output (dig)

```
$ dig +short example.com
93.184.216.34

$ dig example.com
...
;; ANSWER SECTION:
example.com. 3600 IN A 93.184.216.34
example.com. 3600 IN AAAA
2606:2800:220:1:248:1893:25c8:1946
```

DNS Caching

- Resolvers cache DNS records for a period of time (TTL - Time To Live).
- Caching reduces the number of queries and makes resolution faster.
- You can see the TTL value in dig output.
- Records expire after the TTL and are refreshed.

Common DNS Failures

- DNS server unreachable (no internet or ISP issue).
- Domain does not exist (NXDOMAIN).
- Incorrect DNS records.
- DNS caching old records.
- Firewall blocking DNS (port 53).
- DNS misconfiguration.
- High latency or slow DNS responses.

Key Takeaways

- DNS translates names to IP addresses.
- It uses recursive resolvers, root, TLD and authoritative nameservers.
- Common records include A, AAAA and CNAME.
- Resolvers cache results using TTL.
- You can use dig and nslookup to query DNS records.
- DNS failures can be caused by network, configuration, or record issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

10

11 DHCP and Automatic Configuration

What is DHCP?

- DHCP (Dynamic Host Configuration Protocol) automatically assigns IP configuration to devices on a network.
- It gives devices an IP address, subnet mask, default gateway, DNS server and other settings.
- It removes the need for manual configuration.
- Widely used in home networks, corporate networks and cloud environments.

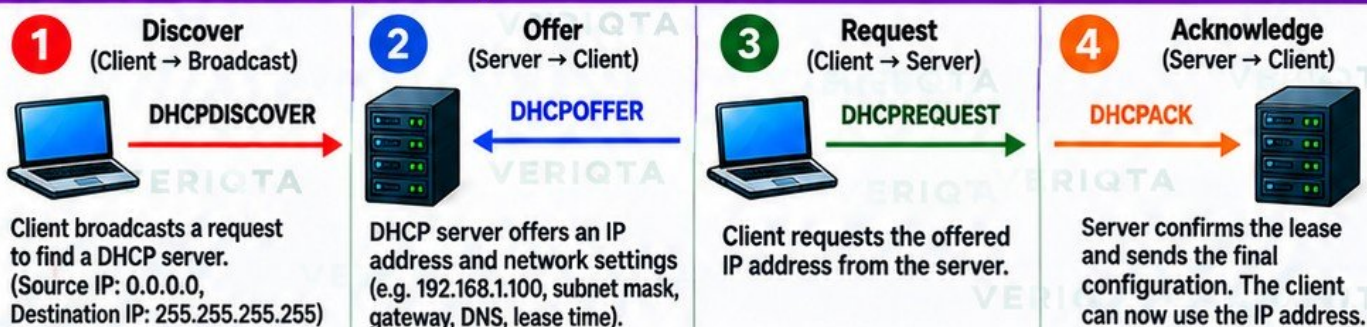
Why Use DHCP?

- Saves time and reduces manual work.
- Prevents IP address conflicts (duplicates).
- Efficient use of IP addresses through leased addresses.
- Simplifies network management.
- Automatically provides gateway and DNS information.
- Devices can move between networks and get the correct configuration.

Key Terms

Term	Meaning
DHCP Server	Assigns IP configuration to clients.
DHCP Client	A device that requests IP configuration.
Lease	Time period the client can use the assigned IP address.
DHCP Scope	Range of IP addresses available for assignment.
Default Gateway	Router IP for reaching other networks.
DNS Server	Used to resolve domain names to IP addresses.

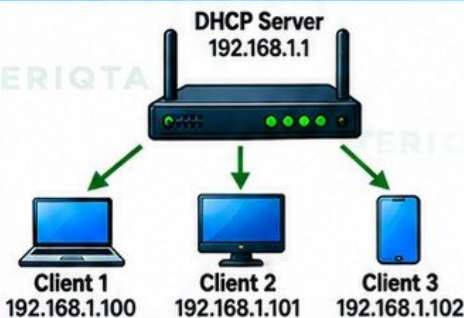
The DHCP Process (DORA)



Information Provided by DHCP

Configuration Item	Example
IP Address	192.168.1.100
Subnet Mask	255.255.255.0
Default Gateway	192.168.1.1
DNS Server	8.8.8.8
Lease Time	86400 seconds (24 hours)
Domain Name (optional)	example.com
Other Options (optional)	NTP server, DNS suffix, etc.

Example DHCP Network



Clients Receive:

- ✓ IP address from DHCP scope
- ✓ Subnet mask
- ✓ Default gateway (router)
- ✓ DNS server
- ✓ Lease time

Check DHCP Configuration (Linux)

```
# View IP configuration (should show DHCP)
ip addr show

# View DHCP lease information (if available)
cat /var/lib/dhcp/dhclient.leases

# View current routes (check default gateway)
ip route

# View DNS configuration
cat /etc/resolv.conf
```

Check DHCP Configuration (Windows)

```
# View IP configuration
ipconfig /all

# Release current DHCP lease
ipconfig /release

# Request a new DHCP lease
ipconfig /renew

# View DNS servers
ipconfig /displaydns
```

Common DHCP Issues

- No IP address assigned.
- APIPA address (169.254.x.x).
- Incorrect gateway or DNS.
- IP address conflicts.
- DHCP server unreachable.
- Lease expiration.
- Client cannot renew lease.

Key Takeaways

- DHCP automatically assigns IP configuration.
- It uses the DORA process (Discover, Offer, Request, Acknowledge).
- Provides IP, subnet mask, gateway, DNS and more.
- Leases allow efficient use of IP addresses.
- Essential for modern networks.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

11

12 NAT and Private Networks

What is NAT?

- NAT (Network Address Translation) changes private IP addresses to a public IP address.
- It allows multiple devices in a private network to access the internet using a single public IP address.
- NAT hides internal private IP addresses from the internet.
- It is widely used in home networks, corporate networks and cloud environments.

Key Terms

- **Private IP Address:** Used inside local networks (not routable on the internet).
- **Public IP Address:** Used on the internet (routable).
- **NAT:** Translates private IPs to a public IP.
- **PAT (Port Address Translation):** NAT with port numbers so multiple devices can share one public IP.
- **Outbound Connection:** Traffic from a private network to the internet (using NAT).

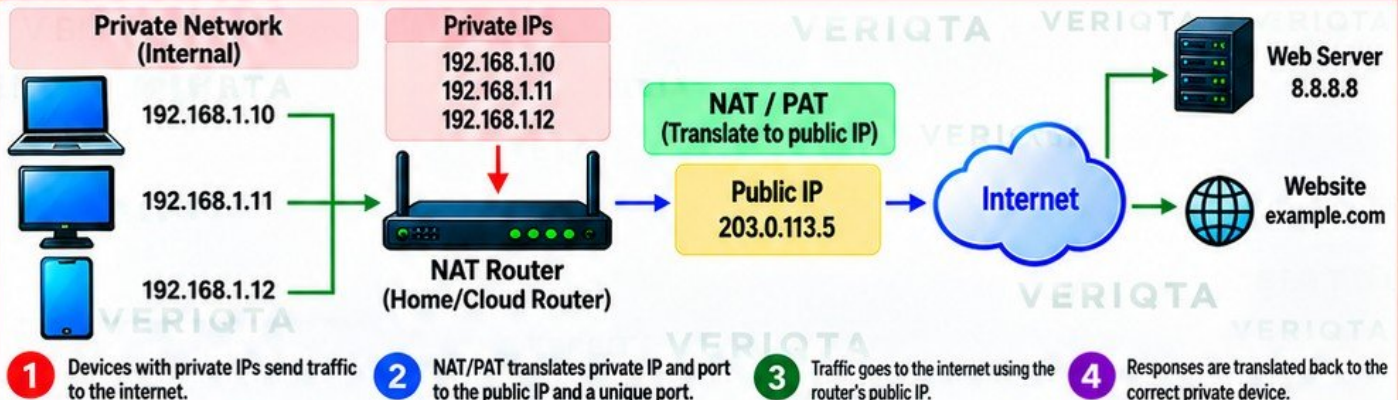
Private IP Address Ranges

Range	Example
10.0.0.0/8	10.0.0.1 – 10.255.255.254
172.16.0.0/12	172.16.0.1 – 172.31.255.254
192.168.0.0/16	192.168.0.1 – 192.168.255.254

Why Use NAT?

- Conserves public IP addresses.
- Allows many devices to share one public IP.
- Provides a layer of security by hiding internal IP addresses.
- Required in home routers, corporate networks and cloud environments.

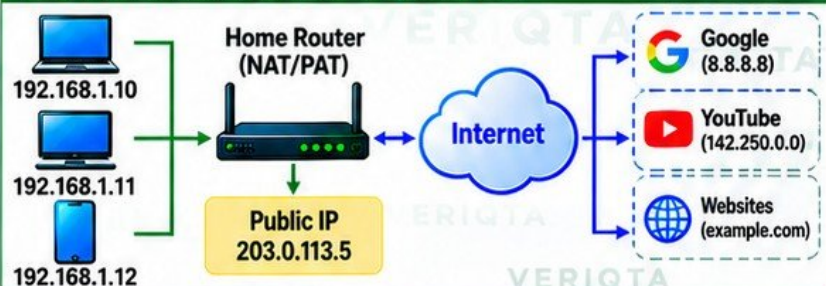
How NAT and PAT Work



NAT vs PAT

Feature	NAT	PAT (NAT Overload)
Translation	Private IP to Public IP	Private IP + Port to Public IP + Port
Public IPs used	One public IP per device	One public IP for multiple devices
Port numbers	Not used	Uses unique port numbers
Use case	Simple translation	Most common (home, cloud, enterprise)

Real-World Example (Home Network)



Check NAT Translation (Linux)

```
# View NAT rules
sudo iptables -t nat -L -n -v

# View active connections (source/private and public)
sudo conntrack -L | grep <private_ip>

# View public IP (from internet)
curl ifconfig.me
```

Check NAT Translation (Cloud - AWS)

```
# VPC uses NAT Gateway for outbound internet
# Check route table (should route 0.0.0.0/0 to NAT)
aws ec2 describe-route-tables \
  --filters "Name=vpc-id,Values=<vpc-id>"

# Check your instance public IP (via NAT)
curl ifconfig.me
```

Key Takeaways

- Private IP addresses are not routable on the internet.
- NAT translates private IPs to a public IP.
- PAT uses port numbers to allow multiple devices to share one public IP.
- NAT is used for outbound connections.
- Home routers, corporate networks and cloud environments use NAT to conserve public IPs and provide security.

13 Firewalls and Network Security

What is a Firewall?

- A firewall controls network traffic based on security rules (allow or deny).
- It sits between trusted and untrusted networks (e.g. your network and the internet).
- It helps block unwanted or malicious traffic.
- Firewalls can be hardware devices, software on a server, or cloud services (e.g. AWS Security Groups).

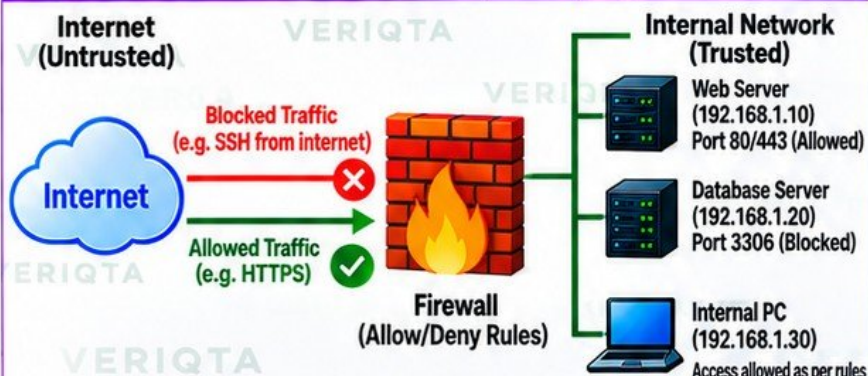
Key Concepts

- **Inbound Traffic:** Traffic coming to your network.
- **Outbound Traffic:** Traffic leaving your network.
- **Allow/Deny Rules:** Rules that decide what traffic is permitted or blocked.
- **Stateful Filtering:** Tracks the state of connections and allows return traffic for established connections.
- **Least Privilege:** Only allow the minimum traffic required.

Why Firewalls are Important

- Reduce the risk of unauthorized access.
- Block malicious and suspicious traffic.
- Protect internal systems and data.
- Control which services are exposed to the internet.
- Help meet security and compliance requirements.
- Used in home networks, corporate networks and cloud environments.

How a Firewall Filters Traffic (Example)



Common Firewall Rules

Rule	Direction	Action	Example
Allow HTTP/HTTPS	Inbound	Allow	Allow port 80, 443 to web server
Allow SSH (from trusted IP)	Inbound	Allow	Allow port 22 from 203.0.113.10
Deny all other inbound traffic	Inbound	Deny	Block everything else
Allow outbound web access	Outbound	Allow	Allow ports 80, 443 to internet
Deny outbound unnecessary traffic	Outbound	Deny	Block all other outbound traffic

Stateful vs Stateless Firewalls

Feature	Stateful Firewall	Stateless Firewall
Tracks connection state	Yes	No
Allows return traffic automatically	Yes	No
More secure	Yes	No
Common use	Modern networks	Simple networks

Security Groups (Cloud Example)



Checking Open Ports (Linux)

```
# Check listening ports
sudo ss -tln

# Check open ports with nmap
nmap -sS example.com

# Check if a port is open
nc -zv example.com 80
```

Commonly Exposed Ports

Port	Service	Risk if Exposed
22	SSH	Brute force attacks
23	Telnet	Insecure (no encryption)
80	HTTP	May be acceptable (web)
443	HTTPS	Usually required (web)
3306	MySQL	Database access (high risk)
3389	RDP	Remote access (high risk)
8080	Web App	Often exposed by mistake

Key Takeaways

- Use a firewall to control inbound and outbound traffic.
- Follow the principle of least privilege (allow only what you need).
- Use stateful firewalls for better security.
- Regularly review and audit your rules.
- Use security groups in cloud environments (e.g. AWS, Azure, GCP).
- Avoid exposing unnecessary ports to the internet.
- Always monitor and log firewall activity.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

13

What is HTTP?

- HTTP (HyperText Transfer Protocol) is the protocol used to transfer web data between a client (browser) and a server.
- It uses a request and response model.
- It is stateless (each request is independent).
- HTTP traffic is not encrypted and can be read by anyone on the network.

What is HTTPS?

- HTTPS is HTTP over TLS (Transport Layer Security).
- It encrypts the data between your browser and the server.
- It protects against eavesdropping, tampering and man-in-the-middle attacks.
- It uses certificates to verify the server's identity.
- It runs on port 443 (HTTP runs on port 80).

What is TLS?

- TLS (Transport Layer Security) provides encryption, authentication and data integrity.
- It ensures the connection is secure before HTTP data is sent.
- It uses certificates, a key exchange process and encryption algorithms.
- TLS is the modern, secure version of SSL (which is now deprecated).
- It protects the data from being read or modified by attackers.

Request and Response Flow (HTTP)



HTTP Methods (Common)

Method	Purpose
GET	Retrieve a resource.
POST	Send data to create a resource.
PUT	Update a resource.
DELETE	Remove a resource.
HEAD	Retrieve headers only.
OPTIONS	Check supported methods.
PATCH	Partially update a resource.

Common HTTP Status Codes

Code	Meaning
200	OK (success)
301	Moved permanently
400	Bad request
401	Unauthorized
403	Forbidden
404	Not found
500	Internal server error
503	Service unavailable

How HTTPS and TLS Work (When You Visit a Secure Site)



Example HTTP Request

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

This request asks the server for /index.html using the GET method.

Example HTTP Response

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1256

<html>
<head>...</head>
<body>...</body>
```

The server responds with status code 200 OK and the HTML content.

Example TLS Handshake (Simplified)

- 1 ClientHello – Browser sends supported TLS versions and encryption algorithms.
- 2 ServerHello – Server selects the TLS version and algorithm.
- 3 Certificate – Server sends its certificate.
- 4 Key Exchange – Client and server agree on a shared secret.
- 5 Finished – Encrypted connection is established.

Check a Certificate (Linux)

```
# View certificate details
echo | openssl s_client -connect example.com:443 \
-servername example.com
```

Test a Website (Linux)

```
# Check HTTP response headers
curl -I http://example.com

# Check HTTPS response headers
curl -I https://example.com
```

Key Takeaways

- HTTP is not encrypted, HTTPS uses TLS.
- TLS provides encryption and server authentication.
- Certificates prove the server's identity.
- Check the URL starts with https and a valid certificate.
- Use common HTTP methods and understand status codes.
- TLS prevents attackers from reading or modifying your data.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

14

What are Linux Networking Commands?

- Linux provides powerful commands to check network configuration, connectivity, routing, open ports, DNS and more.
- These commands help you troubleshoot network issues and understand how your system communicates with other devices.
- They are essential for system administrators, DevOps engineers and anyone working with Linux.

Common Commands Covered

- ip addr** – View and configure IP addresses on network interfaces.
- ip route** – View and manage the routing table.
- ping** – Test network connectivity.
- ss** – Check open ports and active connections.
- curl** – Test HTTP/HTTPS connections.
- dig** – Query DNS records.
- traceroute** – Trace the path to a remote host.

When to Use Each Command

- Use **ip addr** to check your IP address and network interfaces.
- Use **ip route** to see how traffic is routed.
- Use **ping** to check if a host is reachable.
- Use **ss** to see which ports are open and which connections are active.
- Use **curl** to test web services (HTTP/HTTPS).
- Use **dig** to troubleshoot DNS issues.
- Use **traceroute** to see the path packets take to reach a destination.

1. ip addr

View and configure IP addresses on network interfaces.

```
# Show all interfaces
ip addr

# Show a specific interface
ip addr show eth0
```

Use when:

- ✓ You want to see your server's IP address, subnet mask and network interfaces.

2. ip route

View and manage the routing table.

```
# Show routing table
ip route

# Show default route
ip route show default
```

Use when:

- ✓ You want to see how traffic is routed to different networks and the default gateway.

3. ping

Test network connectivity between hosts.

```
# Ping a host
ping example.com

# Send 4 packets
ping -c 4 8.8.8.8
```

Use when:

- ✓ You want to check if a host is reachable and measure response time.

4. ss

Check open ports and active connections.

```
# Show listening ports
ss -tln

# Show all connections
ss -tunap
```

Use when:

- ✓ You want to see which ports are open and which network connections are active.

5. curl

Test HTTP/HTTPS connections and view web responses.

```
# Test HTTP connection
curl http://example.com

# Test HTTPS connection
curl https://example.com
```

Use when:

- ✓ You want to test if a web service is reachable and see the response from a server.

6. dig

Query DNS records.

```
# Query A record
dig example.com

# Query specific record type
dig example.com AAAA

# Use a specific DNS server
dig @8.8.8.8 example.com
```

Use when:

- ✓ You want to check DNS records and troubleshoot DNS resolution issues.

7. traceroute

Trace the path packets take to a destination.

```
# Trace route to a host
traceroute example.com

# Use TCP instead of UDP
traceroute -T example.com
```

Use when:

- ✓ You want to see each hop (network device) between your server and the destination.

Quick Command Comparison

Command	Main Use
ip addr	View IP addresses and interfaces
ip route	View routing table
ping	Test connectivity
ss	View open ports and connections
curl	Test HTTP/HTTPS
dig	Query DNS records
traceroute	Trace network path

Real-World Examples

- Check your server's IP address and default gateway.
- Test if a website is reachable.
- Find out which ports are open on your server.
- Troubleshoot DNS resolution problems.
- Trace the network path to a remote server.
- Test a web API using curl.

Sample Output (ping)

```
$ ping -c 4 example.com
PING example.com (93.184.216.34) 56(84) bytes of data:
64 bytes from 93.184.216.34: icmp_seq=1 ttl=56 time=14.2 ms
64 bytes from 93.184.216.34: icmp_seq=2 ttl=56 time=14.1 ms
64 bytes from 93.184.216.34: icmp_seq=3 ttl=56 time=14.3 ms
64 bytes from 93.184.216.34: icmp_seq=4 ttl=56 time=14.0 ms
--- example.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss,
rtt min/avg/max/mdev = 14.0/14.1/14.3/0.1 ms
```

Key Takeaways

- Use the right command for the right problem.
- These commands help you understand and troubleshoot network issues.
- They are essential for Linux administration and DevOps work.
- Combine commands (e.g., ping, dig, traceroute) to get a complete picture of network issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

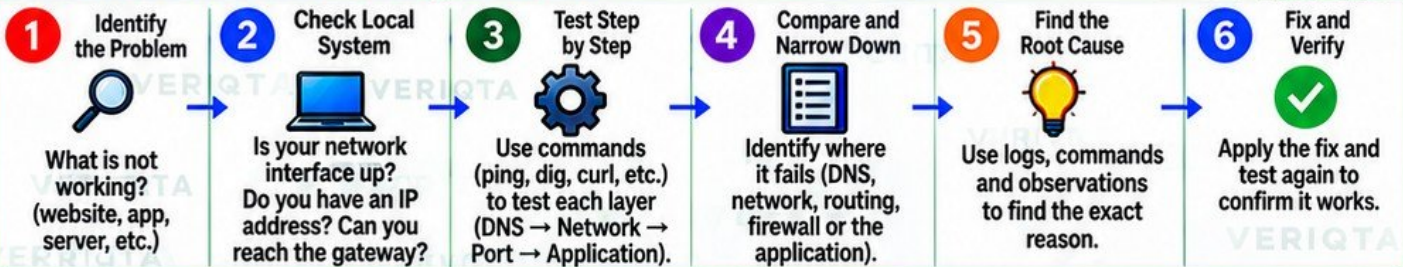


@veriqta

15

16 Troubleshooting Network Failures

Structured Troubleshooting Workflow



1. DNS Failure (Cannot resolve domain name)



Example Error
ping: cannot resolve
example.com:
Name or service
not known

Common Causes

- Wrong DNS server.
- DNS server unreachable.
- Domain does not exist.
- DNS record misconfiguration.

Troubleshooting Commands

```
# Check DNS resolution
dig example.com

# Try another DNS server
dig @8.8.8.8 example.com

# Check your DNS configuration
cat /etc/resolv.conf
```

2. Timeout (No response)



Example Error
ping: sendmsg:
Operation timed out

Common Causes

- Network unreachable.
- Firewall blocking traffic.
- Host is down or not responding.
- Routing issue.

Troubleshooting Commands

```
# Test connectivity
ping example.com

# Try with more details
ping -c 4 -v example.com

# Check the route
traceroute example.com
```

3. Connection Refused (Port closed)



Example Error
curl: (7) Failed to
connect: Connection
refused

Common Causes

- Service is not running.
- Port is closed.
- Firewall is blocking the port.
- Service is listening only on localhost.

Troubleshooting Commands

```
# Check if port is open
ss -tln | grep :80

# Test connection to port
nc -zv example.com 80

# Check service status
sudo systemctl status nginx
```

4. Unreachable Host (No route to host)



Example Error
ping: sendmsg:
No route to host

Common Causes

- No route to the destination.
- Incorrect IP address.
- Network interface down.
- Wrong gateway or routing table.

Troubleshooting Commands

```
# Check your IP and interface
ip addr

# Check routing table
ip route

# Test gateway
ping <gateway_ip>
```

5. Routing Failure (Wrong or missing route)



Example Error
Network is
unreachable

Common Causes

- Missing or incorrect route.
- Wrong default gateway.
- Routing loop or misconfiguration.
- Network change or outage.

Troubleshooting Commands

```
# View routing table
ip route

# Check specific route
ip route get example.com

# Trace the path
traceroute example.com
```

6. Blocked Ports (Filtered by firewall)



Example Error
timeout (no response)
or connection refused

Common Causes

- Firewall blocking the port.
- Security group rules (cloud).
- Network ACL blocking traffic.
- Service not allowed from your IP.

Troubleshooting Commands

```
# Check if port is open
nc -zv example.com 443

# Check with telnet
telnet example.com 443

# Use nmap to scan port
nmap -p 80,443 example.com
```

Example Troubleshooting Flow

1. Can you resolve the domain? (dig)
2. Can you reach the host? (ping)
3. Is the port open? (ss, nc, nmap)
4. Is there a route? (ip route, traceroute)
5. Is the service running? (curl)
6. Check firewalls/security groups
7. Review logs and fix the issue

Common Network Error Messages

Error Message	Meaning
Name or service not known	DNS resolution failed.
Operation timed out	No response (timeout).
Connection refused	Port closed or service not running.
No route to host	No route to destination.
Network is unreachable	Network or route issue.
Connection reset	Connection was closed by the server.

Quick Command Reference

Command	Main Use
ip addr	View IP addresses and interfaces
ip route	View and manage routes
ping	Test network connectivity
ss	Check open ports and connections
curl	Test HTTP/HTTPS services
dig	Query DNS records
traceroute	Trace network path
nmap	Scan for open ports

Key Takeaways

- Troubleshoot step by step (DNS → Network → Port → Application).
- Use the right command for the problem.
- Check both local and remote systems.
- Firewalls and security groups often block traffic.
- Most network issues can be diagnosed with a few simple Linux commands.
- Always verify the fix after making changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

16

17

Cloud Networking Fundamentals



What is Cloud Networking?

- Cloud networking connects resources (VMs, containers, databases, services) in the cloud.
- It provides isolated and secure networks like VPC (AWS), VNet (Azure) or VPC (GCP).
- You control IP addressing, routing, security and traffic flow.
- It allows your applications to communicate with the internet and with each other securely.

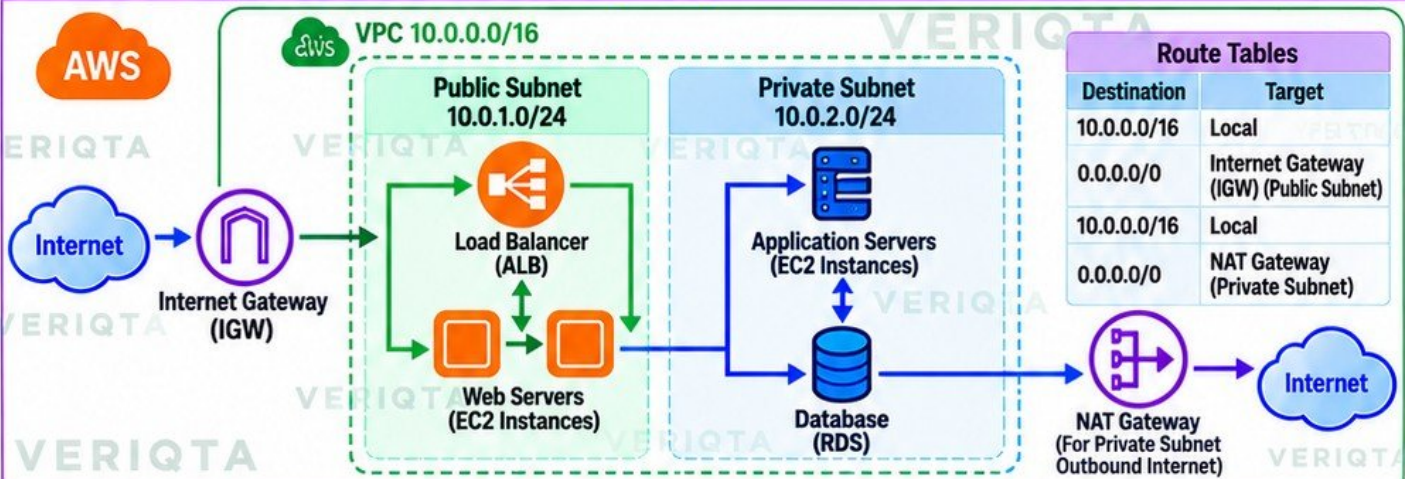
Key Concepts

- VPC/VNet:** Your isolated virtual network in the cloud.
- Subnets:** Smaller networks inside a VPC (public or private).
- Route Tables:** Control where traffic goes.
- Gateways:** Connect your network to the internet or other networks.
- Security Controls:** Security groups, network ACLs, firewalls.
- Load Balancers:** Distribute traffic across multiple instances.

Why It Matters

- Keeps your resources isolated and secure.
- Controls how traffic flows between internet, internal services and other networks.
- Helps you build highly available, scalable and resilient applications.
- Reduces the risk of unauthorized access.
- Essential for running real-world applications in the cloud.

Example Cloud Network Architecture (AWS)



Public vs Private Subnets

Public Subnet	Private Subnet
Has a route to the internet (IGW)	No direct route to the internet
Used for resources that need internet access (e.g. load balancers, web servers)	Used for internal resources (e.g. application servers, databases)
Can have public IPs	Usually no public IPs
More exposed	More secure

Common Cloud Network Components

VPC / VNet (Isolated network)	Security Groups (Allow/deny traffic)
Subnets (Public and private)	Network ACLs (Subnet level filtering)
Route Tables (Control traffic)	Load Balancers (Distribute traffic)
Internet Gateway (IGW) (Access to internet)	VPC Peering (Connect VPCs)
NAT Gateway (Outbound internet for private subnets)	VPN / Direct Connect (Connect to on-premises)

Cloud Traffic Flow (Example)

- 1 User accesses your application on the internet.
- 2 Traffic goes to the Internet Gateway and then to the Load Balancer (in public subnet).
- 3 Load Balancer distributes traffic to web servers (public subnet).
- 4 Web servers communicate with application servers (private subnet).
- 5 Application servers connect to the database (private subnet).
- 6 Private resources use a NAT Gateway for outbound internet access (e.g. updates, external APIs).

Key Security Controls

- Use security groups to allow only required traffic (e.g. HTTP/HTTPS).
- Use Network ACLs for additional subnet level protection.
- Keep databases and sensitive resources in private subnets.
- Follow the principle of least privilege.
- Regularly review and audit your network rules.

Real-World Use Cases

- Hosting a web application (public subnet + ALB).
- Running a database in a private subnet.
- Connecting multiple VPCs across regions.
- Hybrid connections to on-premises networks.
- Running microservices with internal communication.
- Secure access to cloud resources using VPN.

Key Takeaways

- A VPC/VNet gives you an isolated and secure network in the cloud.
- Use public and private subnets to control exposure.
- Route tables and gateways control where traffic goes.
- Security groups and ACLs protect your resources.
- Load balancers distribute traffic and improve availability.
- Understand the full traffic flow from internet to your cloud resources.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

17

Why Networking Matters?

- Containers and Kubernetes use networking to allow applications to communicate with each other and with external services.
- It enables microservices, scalability and high availability.
- Understanding the networking model helps you troubleshoot issues and design secure, reliable applications.

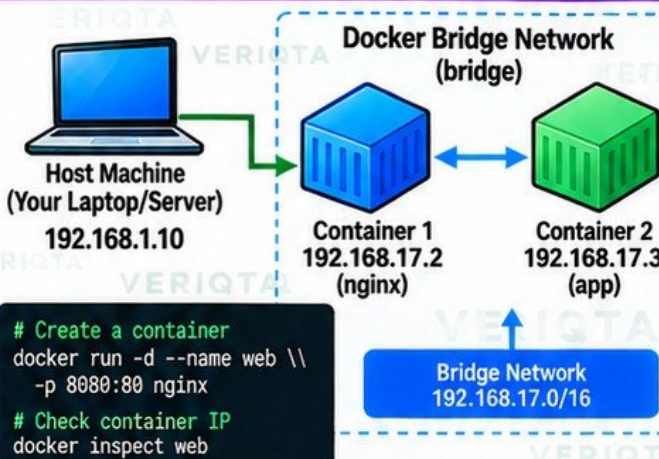
Key Concepts

- Container IP:** Each container gets its own IP address.
- Bridge Network:** Default Docker network for containers on the same host.
- Pod:** Smallest unit in Kubernetes that runs one or more containers (shared IP).
- Service:** Stable network endpoint to access pods.
- DNS:** Service names are automatically resolved inside the cluster.
- Ports:** Control how traffic enters and reaches your application.

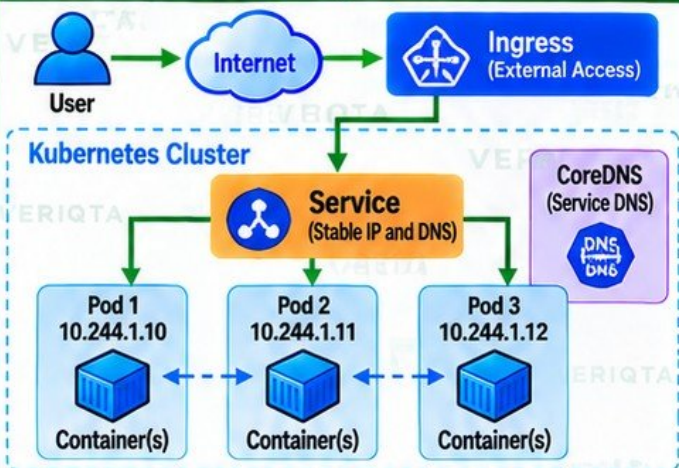
Beginner Mental Model

- Think of a container as a small server with its own IP.
- On your local machine, containers connect using a bridge network.
- In Kubernetes, containers run in Pods, and Pods have their own IP addresses.
- Services provide a stable way to access Pods (even when they restart).
- DNS allows you to use service names instead of IP addresses.
- Ingress or LoadBalancer exposes your application to users outside the cluster.

Docker Container Networking (Bridge Network)



Kubernetes Networking Overview



Kubernetes Service Types

Service Type	What It Does	Use Case
ClusterIP	Internal IP inside cluster (default)	Access between pods only
NodePort	Exposes service on each node's IP at a static port	Testing / simple external access
LoadBalancer	Creates a cloud load balancer (e.g. AWS ELB, Azure LB)	Production external access
ExternalName	Maps to external DNS name	Access external services

Kubernetes DNS Example

Each service gets a DNS name automatically:

```
# Access a service from inside a pod
curl http://my-service.default.svc.cluster.local
```

DNS format:

<service-name>.<namespace>.svc.cluster.local

Example

Service Name: my-service
Namespace: default
DNS Name: my-service.default.svc.cluster.local

Pod to Pod Communication



Ports in Containers and Kubernetes

- Container Port:** Port the application listens on inside the container (e.g. 80).
- Host Port (Docker):** Maps a host port to container port (e.g. -p 8080:80).
- Service Port:** Port exposed by the Kubernetes service.
- Target Port:** Port on the pod/container.
- NodePort / LoadBalancer:** Exposes the service outside the cluster.

Key Takeaways

- Containers have their own IP addresses.
- Bridge networks allow containers to communicate on the same host.
- In Kubernetes, Pods have IPs and can communicate directly.
- Services provide stable access via IP and DNS.
- Use the right service type for your use case.
- DNS makes it easy to access services using names.
- Ingress or LoadBalancer exposes applications to external users.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

18

19

Observability and Packet Analysis



What is Observability?

- Observability helps you understand what is happening in your network and systems.
- It uses metrics, logs and traces to give you visibility.
- It helps you measure latency, packet loss, connection state and overall health.
- You use this information to find and fix problems based on real evidence.

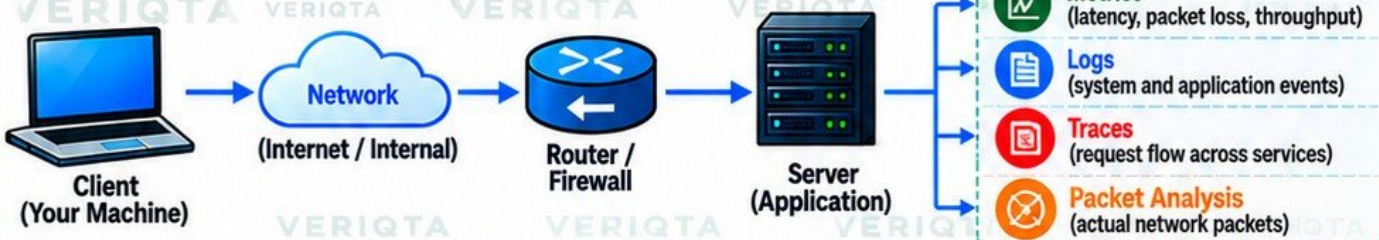
Key Concepts

- **Latency:** Time for a packet to travel from source to destination.
- **Packet Loss:** Packets that do not reach the destination.
- **Connection State:** Current status of a network connection.
- **Logs:** Recorded events from systems, applications and network devices.
- **Network Metrics:** Quantitative data like latency, throughput and errors.
- **Packet Analysis:** Inspecting the actual packets to see what is happening.

Tools We Will Use

- **ping** – Test connectivity and measure latency.
- **mtr** – Check latency and packet loss over time.
- **ss** – View connection state and open ports.
- **netstat** – Alternative to view network connections.
- **tcpdump** – Capture and analyze packets from the command line.
- **Wireshark** – Visual packet analysis tool with detailed inspection.
- **Logs and Metrics** – Use system and application logs, and network metrics from monitoring tools.

How Network Observability Fits Together



1. Measure Latency and Packet Loss

ping

Test basic connectivity and measure latency.

```
# Ping a host
ping example.com

# Send 4 packets
ping -c 4 example.com
```

Use when:

- ✓ You want to quickly check if a host is reachable and see the response time.

mtr

Check latency and packet loss over time.

```
# Run mtr
mtr example.com
```

Use when:

- ✓ You want detailed latency and packet loss information at each hop.

2. Check Connection State

ss

View open ports and connection states.

```
# Show all connections
ss -tuln

# Show established
ss -tan state established
```

Use when:

- ✓ You want to see which ports are open and the state of network connections.

netstat

Alternative to view network connections.

```
# Show all connections
netstat -tuln

# Show established
netstat -tan
```

Use when:

- ✓ You need an alternative to ss (older systems).

3. Capture and Analyze Packets

tcpdump

Capture packets from the command line.

```
# Capture packets
sudo tcpdump -i eth0

# Snpture HTTP traffic
sudo tcpdump -i eth0 port 80
```

Use when:

- ✓ You want to see the actual network packets and troubleshoot issues.

Wireshark

Visual packet analysis with detailed inspection.

```
# Launch Wireshark
(Use GUI)
```

Use when:

- ✓ You want a visual interface to analyze packets in detail.

Example: Analyze a Web Request

- 1 Ping the server to check basic connectivity.
ping example.com
- 2 Use mtr to check latency and packet loss.
mtr example.com
- 3 Check if the port is open.
ss -tuln | grep :80
- 4 Capture packets and filter in Wireshark or tcpdump.
sudo tcpdump -i eth0 port 80
- 5 Review logs and metrics to find the root cause.

Sample tcpdump Output

```
# Capture HTTP traffic
sudo tcpdump -i eth0 port 80

tcpdump: verbose output suppressed
listening on eth0, link-type EN10MB
10:24:12.123456 IP 192.168.1.10.54321 > 93.184.216.34.80: Flags [S], seq 12345, win 64240, length 0
10:24:12.123789 IP 93.184.216.34.80 > 192.168.1.10.54321: Flags [S.], seq 67890, ack 12346, win 65535, length 0
```

Key Takeaways

- Use the right tool for the right problem (ping, mtr, ss, tcpdump, Wireshark).
- Look at latency, packet loss and connection state.
- Analyze packets to see what is really happening on the network.
- Use logs and metrics to get the full picture.
- Always use evidence (data) to diagnose and fix network issues.
- Combining multiple tools gives you better visibility and faster troubleshooting.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

19

Lab Overview

- A realistic broken application where you trace the full path from Client → DNS → Network → Firewall → Server → Application.
- You will identify the exact failure, prove the root cause with evidence, and fix the issue.
- This lab builds real troubleshooting skills used in production by system administrators, DevOps and cloud engineers.

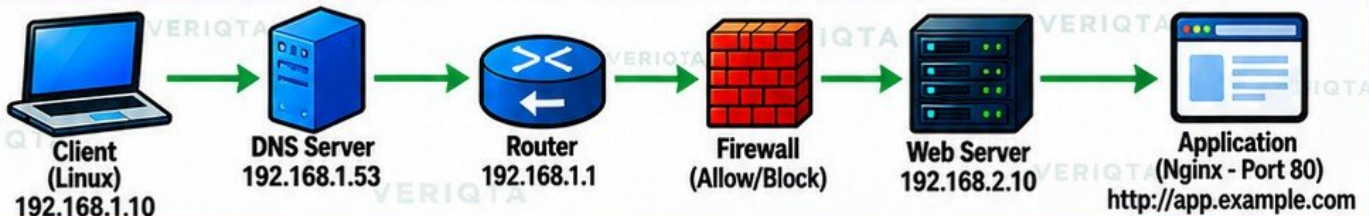
Lab Environment

- A client machine (Linux).
- A DNS server (internal).
- A network with a router/firewall.
- A web server (Linux) running a sample application (Nginx).
- One deliberate failure is configured (e.g. DNS issue, blocked port, routing problem or application down).
- You will use real Linux networking commands to find and fix the issue.

What You Will Learn

- How to follow a structured troubleshooting workflow.
- How to test each component in the path.
- How to use common Linux networking commands.
- How to interpret the results and find the real root cause.
- How to fix the issue and verify the application is working.
- Build confidence troubleshooting real-world network problems.

Network Architecture (Client → DNS → Network → Firewall → Server → Application)



1. Test from the Client

Start by testing the application from the client.

```
# Try to access the application
curl -v http://app.example.com

# Or test with ping (if allowed)
ping app.example.com
```

Expected:

✗ Application is not reachable (or shows an error).

2. Check DNS Resolution

Verify if the domain name resolves to an IP address.

```
# Query DNS
dig app.example.com

# Or use nslookup
nslookup app.example.com
```

Expected:

✓ You should get the correct IP address (e.g. 192.168.2.10).

3. Test Network Connectivity

Check if you can reach the server IP address.

```
# Test with ping
ping 192.168.2.10

# Trace the network path
traceroute 192.168.2.10
```

Expected:

✓ You should see replies and the correct network path.

4. Check Firewall Rules

Verify if the firewall is allowing the required traffic.

```
# Test if port 80 is open
nc -zv 192.168.2.10 80

# Check firewall rules
sudo iptables -L -n

# or: sudo firewall-cmd --list-all
```

Expected:

✗ You may see that the port is blocked (connection refused or timed out).

5. Check the Server

Verify if the server is reachable and the application is running.

```
# SSH to the server
ssh user@192.168.2.10

# Check service status
sudo systemctl status nginx
```

Expected:

✓ Nginx should be running and listening on port 80.

6. Test the Application

Once networking is working, test the application directly on the server.

```
# Test locally on the server
curl http://localhost

# Check listening ports
ss -tuln | grep :80
```

Expected:

✓ You should see the application response (e.g. Nginx default page).

7. Identify the Root Cause

Use the evidence to find the exact failure.

Common Root Causes:

- DNS record missing or incorrect
- Network routing issue
- Firewall blocking the port
- Server not running
- Application listening on wrong port
- Security group / ACL blocking traffic

8. Fix and Verify

Apply the fix and test again from the client.

```
# Example: allow port 80
sudo firewall-cmd --add-port=80/tcp
sudo firewall-cmd --reload

# Test again
curl -v http://app.example.com
```

Expected:

✓ Application is reachable and working correctly.

Sample Troubleshooting Flow



Example Final Result (Success)

```
$ curl -v http://app.example.com
< HTTP/1.1 200 OK
< Server: nginx/1.24.0
< Content-Type: text/html
<
<html>
<h1>Welcome to VERIQTA Lab</h1>
</html>
```

Key Takeaways

- Follow a structured approach.
- Test each component in the path.
- Use the right commands.
- Look at real evidence (not guesses).
- Identify and fix the actual root cause.
- Verify the application is working.
- These are the same skills used in real production environments.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

20